Transporte Urbano

Julio Garcia Ortiz

1. Función para transformar. 2
2. Función de comunicación Python a Prolog. 3
3. Regla Bicicleta. 3
4. Regla de Autobús. 3
5. Preguntitas 4

1. Función para transformar.

Convertimos mediante código python la distancia de kilómetros a la medida de prolog (corta, media o larga) y hacemos lo mismo con el tiempo transformando los minutos a medidas como (poco, normal o suficiente) para después poder usarlo en las reglas de prolog.

```python
def clasif_distancia(km):
    if km < 2:   return "corta"
    if km <= 10: return "media"
    return "larga"

def clasif_tiempo(min):
    if min < 15:  return "poco"
    if min <= 45: return "normal"
    return "suficiente"
```

2. Función de comunicación Python a Prolog.

Mediante la siguiente función limpiamos los hechos previos para no tener errores o duplicados, insertamos datos del caso actual, consultamos las reglas y cargamos las recomendaciones que obtenemos de prolog mediante las reglas que se le asignaron.

```python
def consultar(dist, tiempo, trafico, clima, presupuesto, preferencia):
    for h in ["distancia","tiempo","trafico","clima","presupuesto","preferencia"]:
        prolog.retractall(f"{h}(_)")
    prolog.assertz(f"distancia({dist})")
    prolog.assertz(f"tiempo({tiempo})")
    prolog.assertz(f"trafico({trafico})")
    prolog.assertz(f"clima({clima})")
    prolog.assertz(f"presupuesto({presupuesto})")
    prolog.assertz(f"preferencia({preferencia})")
    return list(prolog.query("recomienda(Transporte, Prioridad)"))
```

3. Regla Bicicleta.

Mediante este regla prolog prueba las distintas opciones mediante una puerta OR hasta encontrar el caso idóneo que aquí en concreto seria distancia corta y buen tiempo y nos devolverá la recomendación.

```
% ── BICICLETA ──────────────────────────────────────────────
% Distancia corta + buen clima + cualquier preferencia razonable
recomienda(bicicleta, ecologico) :-
    distancia(corta),
    clima(bueno),
    (preferencia(ecologico) ; preferencia(economia) ;
     preferencia(rapidez)   ; tiempo(suficiente)).
```

4. Regla de Autobús.

En este caso vemos una regla que afecta al metro, nos recomienda autobús ya que el presupuesto es bajo (económico) y el clima es extremo. Si la distancia fuera larga y el presupuesto siguiera siendo económico nos recomendaría el metro ya que sería lo más óptimo. Una regla bastante sencilla.

```
% Clima extremo sin presupuesto alto (taxi no accesible)
recomienda(autobus, economico) :-
    clima(extremo),
    \+ presupuesto(alto).
```

5. Preguntitas

1. Explica brevemente qué problema resuelve el sistema desarrollado.
   Conseguimos que el sistema nos recomiende un medio de transporte adecuado, dependiendo de variables como distancia, tiempo, tráfico, clima... por lo que nos dice si mejor coche, autobús, metro, etc.

2. ¿Qué papel desempeña Python dentro del sistema?
   Gestiona la entrada de los datos del CSV, validando que el formato sea correcto y se comunica con prolog para presentar los datos al usuario relativos a las reglas definidas en el . pl

3. ¿Qué papel desempeña PROLOG dentro del sistema?
   Como dije justo arriba, el archivo de prolog contiene las reglas o lógicas. Recibimos los hechos y este archivo devuelve la recomendación.

4. ¿Por qué no se ha implementado todo en un solo lenguaje?
   Porque como hemos visto en otras entregas anteriores prolog es lo mas optimo para definir reglas lógicas y claras.

5. ¿Qué ventajas tiene separar la base de conocimiento del control del sistema?
   Pues que al ser archivos diferentes, puedo cambiar las reglas sin necesidad de editar o ajustar cada vez que necesitemos el archivo python. Esto nos facilita el mantenimiento.

6. ¿Cómo se leen los datos desde el fichero CSV?
   Lo leemos desde la clase de python, accedemos a la ruta del archivo usando la extensión OS y con la extensión csv.DictReader leemos cada fila del fichero.

7. ¿Qué validaciones se realizan sobre los datos de entrada?
   Verificamos que la distancia siempre sea un número mayor que cero, que el tiempo también sea entero y mayor a cero y que los campos categóricos como tráfico, clima o presupuesto también sean válidos.

8. ¿Cómo se transforman los datos numéricos en hechos simbólicos?
   Mediante funciones que convierten el número a una categoría discreta que prolog entiende.

9. Pon un ejemplo concreto de transformación (ej: distancia → distancia_corta).
   distancia_km = 16.2 → clasif_distancia(16.2) → "larga"

10. ¿Qué problemas podrían surgir si los datos no están bien formateados?
    Si algún campo falla o no coincide el valor categórico con el valor válido se produciría un error de sintaxis.

11. ¿Cómo se representan los hechos en PROLOG?
    Como predicados simples: distancia(larga)., trafico(alto).. Son dinámicos: se añaden con assertz y se eliminan con retractall en cada caso.

12. ¿Cómo se representan las reglas del sistema experto?
Con la forma cabeza :- cuerpo. (SI cuerpo ENTONCES cabeza). El cuerpo combina condiciones con , (AND) y ; (OR).

13. Escribe una regla completa en PROLOG utilizada en tu sistema.

```
recomienda(metro, rapido) :-
  (trafico(alto) ; distancia(larga)),
  \+ clima(extremo).
```

14. ¿Qué ventajas tiene PROLOG para representar reglas frente a Python?
Las reglas son declarativas, más legibles y el backtracking automático encuentra todas las soluciones posibles.

15. ¿Qué limitaciones tiene PROLOG en comparación con Python?
No está pensado para leer ficheros, hacer interfaces o gestionar errores, por eso se combina con python.

16. Explica cómo PROLOG realiza el proceso de inferencia.
Busca las reglas cuya cabeza coincida con la consulta, comprueba si el cuerpo se satisface con los hechos disponibles y hace backtracking si alguna condición falla para saltar a la siguiente regla.

17. ¿Qué tipo de razonamiento utiliza PROLOG por defecto?
Por defecto prolog utiliza un razonamiento deductivo con encadenamiento hacia atrás, parte de la conclusión y busca los hechos que la justifican.

18. ¿Qué ocurre cuando varias reglas pueden aplicarse al mismo tiempo?
Aplica todas en orden de aparición y devuelve cada solución encontrada.

19. ¿El sistema puede generar múltiples recomendaciones? ¿Cómo?
Si, devuelve todas las soluciones que prolog encuentra y muestra la primera como principal y el resto como alternativas.

20. ¿Qué sucede si no se cumple ninguna regla?
La consulta devuelve la lista vacía y el sistema muestra que no hay recomendación por lo que continúa a la siguiente consulta sin lanzar error.

21. ¿Cómo se comunican Python y PROLOG en tu implementación?
Usando la librería pyswip, con esta actúa como puente entre python y prolog.

22. ¿Qué dificultades has encontrado en esta comunicación?
Al principio me dio varios errores relacionados con la ruta de los archivos, para esto use la librería de import os para obtener las rutas correctas de los archivos. También

es necesario implementar retractall para limpiar los hechos entre casos para evitar su acumulación.

23. ¿Cómo se envían los hechos desde Python a PROLOG?
Con prolog retractall para limpiar y prolog assertz para añadir el nuevo hecho. Se hace para cada variable del caso.

24. ¿Cómo se recuperan las conclusiones desde PROLOG a Python?
Mediante list(prolog.query), esto nos devuelve una lista de diccionarios donde las claves son las variables y los valores las soluciones encontradas.

25. ¿Qué ocurriría si PROLOG devuelve múltiples soluciones?
El sistema usa la primera como recomendación y el resto como alternativa, tal y como dije anteriormente.

26. ¿Cómo se presentan los resultados al usuario?
En una tabla, mostrando todos los datos y la recomendación principal y justificación.

27. ¿El sistema explica por qué toma una decisión? ¿Cómo?
Si mediante el predicado justification, que almacena un texto explicativo por cada medio de transporte. Este se muestra junto a la recomendación.

28. ¿Qué información se muestra en la salida final?
Número de caso, datos de entrada, medio de transporte recomendado junto a su prioridad, la justificación y las alternativas.

29. ¿Cómo mejorarías la explicación del sistema?
Podría mostrar la regla concreta que se disparó y que condiciones específicas del caso la activaron, para de esta manera entender mejor el funcionamiento.

30. ¿Por qué es importante que un sistema experto sea explicable?
Para que el usuario confíe, de esta forma podrá detectar errores en las reglas y ajustar el comportamiento del sistema. Sin explicación el sistema es más complicado de entender.

31. Describe un caso en el que el sistema recomiende bicicleta.
Caso 31: distancia=1.9km [corta], clima=bueno, preferencia=economia. Se cumple la regla de bicicleta por distancia corta + buen clima + preferencia económica.

32. Describe un caso en el que el sistema recomiende metro.
Caso 01: distancia=16.2km [larga], trafico=alto, clima=lluvia.
Se activa recomienda(metro, rápido) porque el tráfico es alto y la distancia es larga.

33. Describe un caso ambiguo (más de una posible solución).

Caso 04: distancia=larga, trafico=alto, presupuesto=alto. Activa reglas de metro (por tráfico alto y por distancia larga) y de coche compartido (distancia larga + presupuesto alto), generando varias recomendaciones simultáneas.

34. ¿Has encontrado algún caso en el que el sistema falle?
Sí, los casos 74 y 76 inicialmente no obtenían recomendación porque la combinación distancia media + tráfico bajo/medio + clima bueno no estaba cubierta. Solucioné este problema creando una regla de autobús para este caso.

35. ¿Cómo validarías que el sistema funciona correctamente?
Asegurándome de que todos los casos tienen respuesta, osea asegurandome a mano de que todas las opciones tienen regla creada.

36. ¿Qué partes del sistema han sido más fáciles de implementar en PROLOG?
Definir las reglas de recomendación.

37. ¿Qué partes han sido más fáciles en Python?
La lectura de CSV y mostrar los resultados, ya que ya lo hicimos en otras entregas.

38. ¿Qué lenguaje consideras más adecuado para reglas? ¿Por qué?
Como ya dije también anteriormente después de haber buscado un poco de información y preguntarle a ia entendi que el más óptimo para esto es prolog, ya que permite expresar las reglas de forma bastante clara y hace el backtracking automático.

39. ¿Qué lenguaje consideras más adecuado para interacción con el usuario?
Aquí complementarias con python, ya que con este podemos realizar de forma sencilla la carga de datos o limpieza de forma sencilla como ya hicimos en otras actividades.

40. ¿Qué ventajas aporta el enfoque híbrido frente a uno puramente procedural?
Pues principalmente la modularidad que nos ofrece, separando la lógica del control y pudiendo modificar las reglas sin que afecte al código.

41. ¿Qué limitaciones tiene el sistema actual?
Pues que las categorías son fijas y el modelo en si no aprende, simplemente responde a lo que se le enseñó mediante las reglas y si falto alguna regla por añadir podría quedar algún caso sin respuesta concreta o precisa.

42. ¿Qué ocurriría si el número de reglas aumenta mucho?
El mantenimiento se complicaría ligeramente ya que podríamos tener reglas duplicadas o reglas que se contradicen unas a otras.

43. ¿Cómo se podrían gestionar conflictos entre reglas?
Principalmente asignando prioridad a las reglas para que sigan un orden.

44. ¿Cómo podrías añadir incertidumbre (probabilidades)?

Añadiendo un factor de confianza numérico a cada regla que python pondera antes de mostrar la recomendación.

45. ¿Cómo ampliarías el sistema para hacerlo más realista?

Pues se podría conectar con una api para controlar el tiempo, otra para el tráfico, incluso google para cálculo de rutas y mejorando la interfaz para un uso más claro y sencillo.

46. ¿Qué parte del ejercicio te ha resultado más compleja?

Integrar python con prolog, ya que al realizar la tarea en mi portátil personal, al intentar instalar la librería pyswip me dio varios errores.

47. ¿Qué parte te ha resultado más sencilla?

Las reglas de prolog.

48. ¿Qué has aprendido sobre sistemas expertos?

Que al mantener prolog separado dep python podemos tener un sistema más óptimo y sencillo de mantener y que es importante definir de forma clara las reglas para mas adelante no tener errores o problemas.

49. ¿Qué has aprendido sobre PROLOG que no sabías antes?

No sabía que se podría integrar con python de esta manera y ofrece un abanico bastante amplio para sistemas expertos o incluso poder crear un sistema enorme e ir separando distintos archivos de prolog para distintos tipos de reglas si es que tenemos un sistema de tamaño grande que lo pueda soportar o sea necesario.

50. ¿En qué situaciones reales aplicarías un sistema como este?

En sistemas que requieran de conocimientos claros donde pueda declarar bien las reglas para tener un sistema sólido y seguro a la vez que de gran volumen, como para cálculo de rutas para una empresa logística, diagnóstico médico que responde ante distintos síntomas...